EXPERIMENT – 7

Performing Code Injection on Application Process Using Ptrace in Kali Linux

Procedure

Step 1: Start Kali Linux Virtual Machine

1. Open Oracle VirtualBox.
2. Start the Kali Linux virtual machine.
3. Log in using Kali credentials.

Step 2: Create a Target Program

1. Open Terminal.
2. Create a simple C program:
3. nano target.c
4. Write a basic infinite loop program.

```
#include <stdio.h>


int main() {
    while(1) {
        printf("This is an infinite loop...\n");
    }
    return 0;
}
```

5. Compile the program:
6. gcc target.c -o target
7.

Step 3: Run the Target Application

1. Execute the program:
2. ./target
3. Note the **Process ID (PID)** using:
4. ps aux | grep target

Step 4: Create Injector Program

1. Create injector source file:
2. nano injector.c

3. Write ptrace-based code to attach to the target process and modify its memory or registers.

```
#include <stdio.h>
#include <stdlib.h>
#include <sys/ptrace.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <sys/user.h>
#include <unistd.h>

int main(int argc, char *argv[]) {

    if (argc != 2) {
        printf("Usage: %s <pid>\n", argv[0]);
        return 1;
    }

    pid_t pid = atoi(argv[1]);
    struct user_regs_struct regs;

    // Attach to target process
    if (ptrace(PTRACE_ATTACH, pid, NULL, NULL) == -1) {
        perror("ptrace attach failed");
        return 1;
    }

    // Wait for process to stop
    waitpid(pid, NULL, 0);
    printf("Attached to process %d\n", pid);

    // Get registers
    if (ptrace(PTRACE_GETREGS, pid, NULL, &regs) == -1) {
```

```
        perror("getregs failed");
        return 1;
    }

    printf("Original RIP: %llx\n", regs.rip);

    // Modify instruction pointer (example: +2 bytes)
    regs.rip += 2;

    if (ptrace(PTRACE_SETREGS, pid, NULL, &regs) == -1) {
        perror("setregs failed");
        return 1;
    }

    printf("Modified RIP: %llx\n", regs.rip);

    // Detach process
    ptrace(PTRACE_DETACH, pid, NULL, NULL);
    printf("Detached from process\n");

    return 0;
}
```

4. Save the file.

Step 5: Compile the Injector Program

1. Compile the injector:
2. gcc injector.c -o injector

Step 6: Attach to Target Process

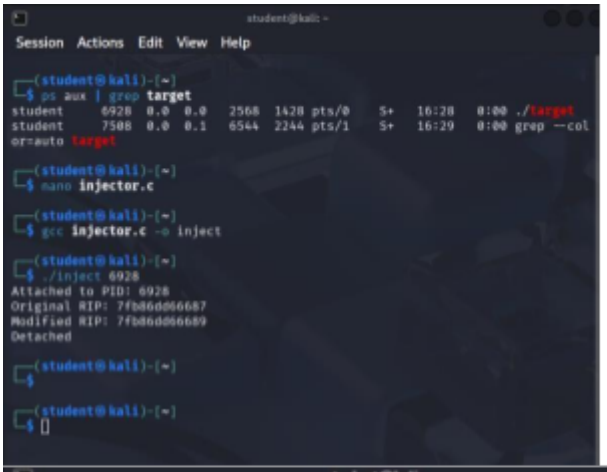
1. Attach injector to target process using PID:
2. sudo ./injector <PID>
3. Injector gains control of the target process.

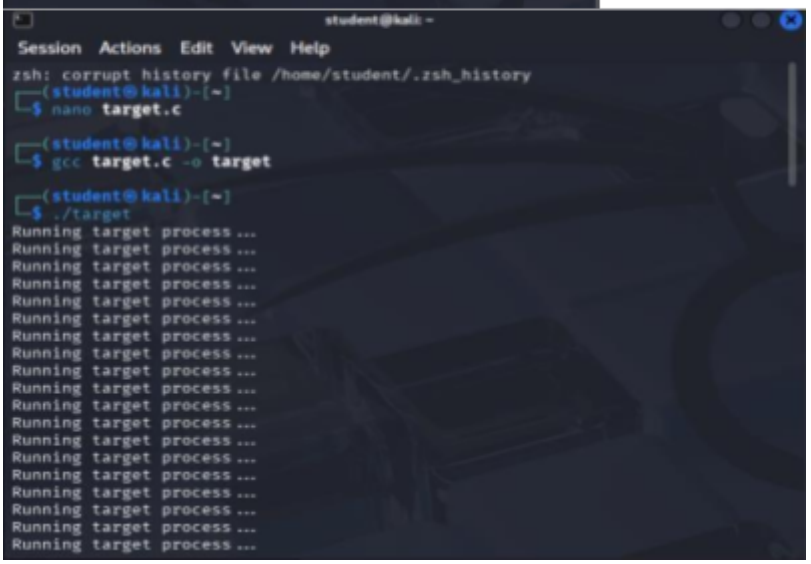
Step 7: Inject and Execute Code

1. Modify target process memory or registers.

2. Resume execution of the target process.
3. Observe injected behavior.

Output





Result

Code injection using the ptrace system call was successfully demonstrated in Kali Linux. The experiment illustrates how process memory and execution flow can be manipulated, emphasizing the importance of secure process isolation.

Reference

https://man7.org/linux/man-pages/man2/ptrace.2.html